

RV32I core architecture

A readable guide to the multicycle controller, datapath, memory ports, and trap boundary

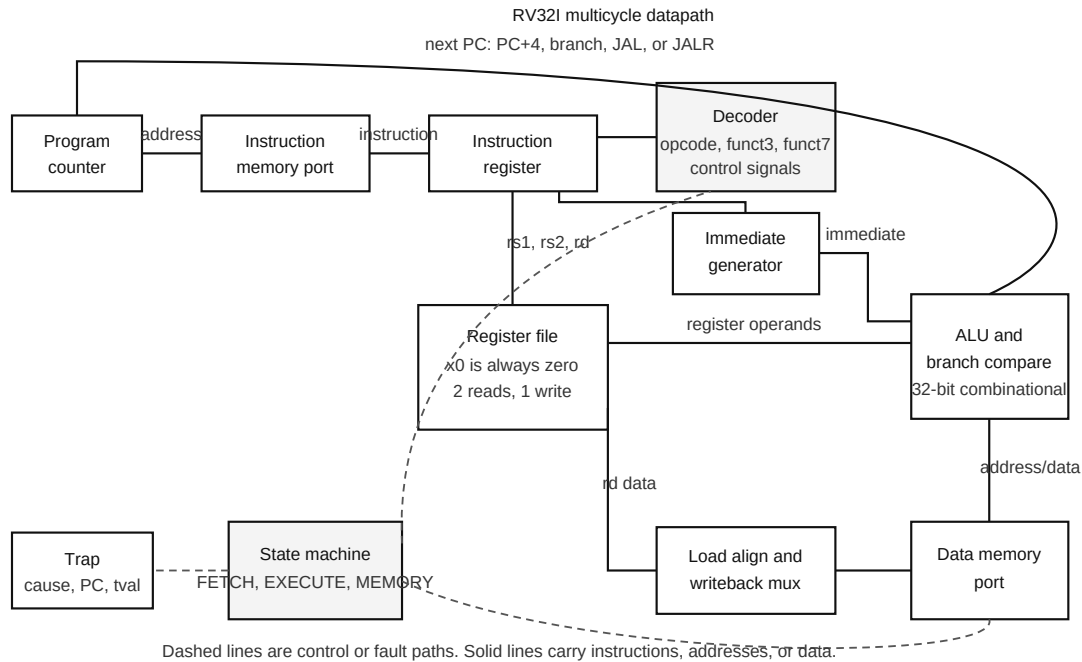


Figure 1. Main datapath and control paths.

Design scope

This core implements the complete unprivileged RV32I base instruction set. It has separate instruction and data memory ports, a 32-register file, alignment checks, and a sticky external trap record. It does not implement compressed instructions, multiplication, privileged CSRs, interrupts, caches, or an MMU.

The design favors readable behavior over peak throughput. Each instruction completes before the next one begins, so there are no pipeline hazards to hide the basic movement of data.

Why the core is multicycle

A single-cycle CPU asks one clock period to cover instruction memory, decode, register reads, the ALU, data memory, and writeback. That drawing is useful in an introductory lesson, but it assumes combinational memories and creates a long timing path.

This core separates the work into states. The state machine is small enough to understand without pipeline hazards, and the memory ports can wait for a real response.

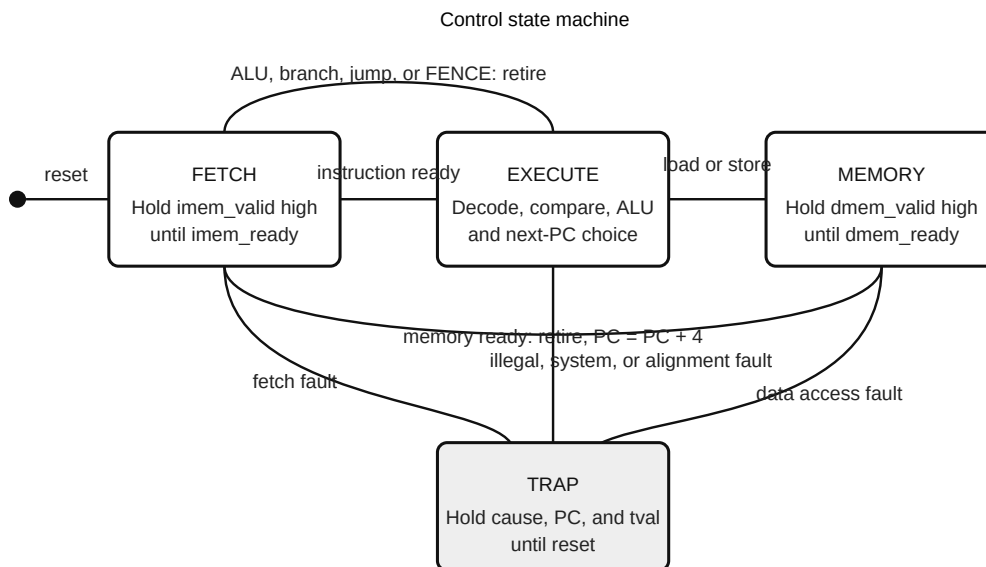


Figure 2. Four-state control machine.

Ordinary instruction sequence

- FETCH holds `imem_valid_o` and the PC until instruction memory raises `imem_ready_i`.
- EXECUTE decodes the saved instruction, reads registers, computes the result, and chooses the next PC.
- MEMORY is used only by a load or store and waits for `dmem_ready_i`.
- TRAP holds the fault record until reset.

Datapath blocks

Program counter and instruction register

The program counter is a byte address. Because the compressed extension is absent, every instruction is four bytes and legal instruction addresses satisfy $pc[1:0] == 2'b00$. The fetched instruction stays in a register while decode and execution finish.

Decoder and immediate generator

The decoder checks the opcode, funct3, and funct7 fields. Reserved encodings become illegal-instruction traps. The immediate generator reconstructs I, S, B, U, and J layouts. Branch and jump outputs already contain their low zero bit, so they are ready to add to the PC.

Register file

Two combinational read ports supply the usual two source operands. One clocked write port records the result. Register x0 is protected twice: reads return zero explicitly, and sequential logic forces its stored value to zero.

ALU and branch comparison

The ALU performs addition, subtraction, shifts, bitwise logic, and signed or unsigned set-less-than operations. The nearby branch comparator uses signed or unsigned interpretation according to the branch instruction.

Writeback

The writeback path selects an ALU result, a completed load, or $PC + 4$ for a jump link. A faulting instruction never writes its destination register.

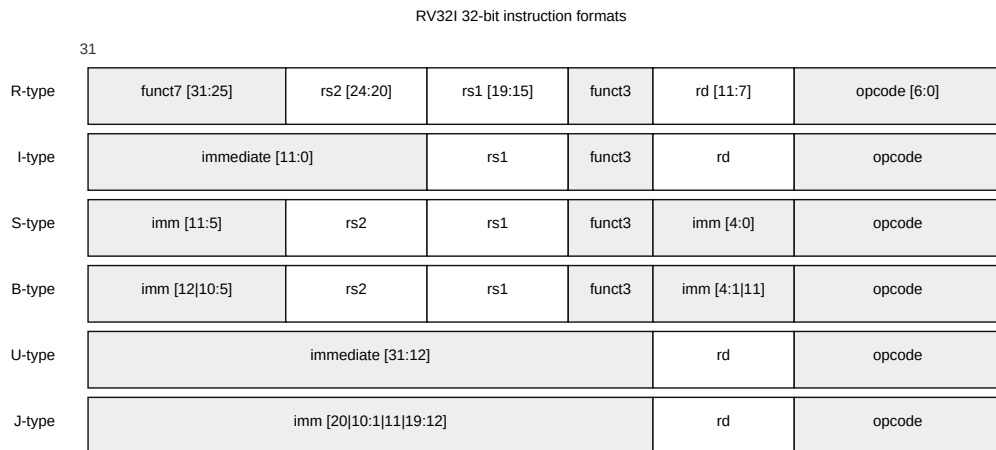


Figure 3. Field placement in the six RV32I instruction formats.

Control transfers

JAL adds a J-type immediate to the current PC. JALR adds an I-type immediate to rs1, then clears target bit 0 as required by the ISA. Bit 1 is still checked because this core requires four-byte instruction alignment.

A taken branch checks its target alignment. A branch that is not taken does not trap because its target is not used. A faulting JAL or JALR does not write its link register.

Loads and stores

Data addresses are byte addresses. Misaligned halfword and word accesses trap instead of being split into several transfers.

Access	Address bits	Write strobe
SB	00, 01, 10, or 11	0001, 0010, 0100, or 1000
SH	00 or 10	0011 or 1100
SW	00	1111

Memory returns the aligned 32-bit word containing the requested bytes. The load-align block shifts the chosen byte or halfword into the low bits, then sign extends LB and LH or zero extends LBU and LHU.

Memory handshake

The core raises `valid` and keeps the address and controls stable. Memory completes the request by raising `ready`. Read data and the error flag are sampled only when both signals are high. Any number of wait cycles is allowed, but the core permits only one outstanding request.

Because no request queue, cache, or write buffer exists, every earlier memory operation has completed when FENCE executes. The core therefore retires FENCE without another hardware action.

Traps

This repository does not implement the privileged architecture. A trap stops the state machine instead of redirecting the PC to a machine-mode handler.

Output	Meaning
trap_valid_o	Sticky high until reset
trap_cause_o	Standard exception number
trap_pc_o	Faulting instruction or failed fetch address
trap_tval_o	Bad instruction word or faulting address when useful

Reset and retirement

`rst_ni` is asynchronous and active low. Reset clears the register file, trap record, and request state, then loads the `RESET_PC` parameter.

`retire_valid_o` pulses when an instruction completes without a trap. `retire_pc_o` and `retire_instruction_o` identify that instruction. These are waveform and scoreboard signals, not the full RISC-V Formal Interface.

Integration limits

- No machine-mode CSRs, trap vector, interrupts, or MRET.
- No M, C, A, Zicsr, or Zifencei extensions.
- Misaligned data is trapped rather than repaired in hardware.
- The native memory ports need a wrapper before connection to AXI or Wishbone.